Università degli Studi di Trento Corso di Programmazione I – E3: 145430/145935 1° Appello – 11/01/2019 – a.a. 2018/2019	**PC N°**
COGNOME ..... **NOME .....** **MATRICOLA** | | | | | | |	

Completare le voci **Cognome**, **Nome** e **Matricola**, se tali voci saranno mancanti o non-leggibili elaborato **NON** verrà corretto.
NON è consentito utilizzo di alcun strumento cartaceo, elettronico, informatico etc. durante la prova.
Fa parte della valutazione la leggibilità del codice C/C++.
Scrivere le risposte su questo foglio e riconsegnare. I risultati verranno pubblicati su DOL
Tempo a disposizione: 120 minuti.

PARTE TEORICA: 12 PUNTI – PUNTEGGIO MINIMO 6 PUNTI

[Domanda 1 – punti 5] Data la seguente definizione di struttura di tipo albero:

```
struct tree { int dato; tree *sx, *dx; } tree;
```

Scrivere 2 funzioni che controllano se c'è almeno un nodo che contien un valore positivo (>0) in un albero binario con radice `root`:

A. Se l'albero NON è stato creato in modalità BST `bool esiste(tree* root);`
B. Se l'albero è un BST `bool esisteBST(tree* root);`

Calcolare la funzione T(N) del tempo di esecuzione per il caso A e B.

[Domanda 2 - punti 2] Spiegare cosa si intente don il termine “Garbage Collection”

[Domanda 3 - punti 3] Risolvere la seguente equazione di ricorrenza, specificando il limite asintotico superiore.

$$\begin{cases} T(1) = N \\ T(N) = 2*T(N/2) \end{cases}$$

[Domanda 4 - punti 2] Date le seguenti dichiarazioni

```
typedef TipoDato *DatoPtr;
TipoDato *puntatore, **doppioPuntatore, x, y;
DatoPtr p, q;
```

Stabilire se le seguenti assegnazioni sono corrette o sbagliate

☐ Vero ☐ Falso `puntatore = &y;`
☐ Vero ☐ Falso `doppioPuntatore = &p;`
☐ Vero ☐ Falso `p = &x;`
☐ Vero ☐ Falso `puntatore = p;`
☐ Vero ☐ Falso `q = p;`
☐ Vero ☐ Falso `*p = *q;`
☐ Vero ☐ Falso `puntatore = doppioPuntatore;`

PARTE PRATICA: 21 PUNTI – PUNTEGGIO MINIMO 11 PUNTI

[A - punti 3] *Inserire i seguenti tipi in un file* ***dati.h*** **e implementare i metodi di stampa ed i costruttori:**

```
#define DIM 2
typedef enum Tprodotto { MUSICA, LIBRI, TECH } Tprodotto;
typedef struct Tacquisto{
  char nomeProdotto[20];
  unsigned int quantita;
  float prezzo;
  Tprodotto tipoProdotto;
  Tacquisto ()  { /*  da implementare */ }
  void stampa() { /*  da implementare → formato: vedi punto E */ }
} Tacquisto;
typedef struct Tnodo {
   Tacquisto dato;
   Tnodo* next;
   Tnodo () { /*  da implementare */ }
   Tnodo (Tacquisto a, Tnodo* n) { /*  da implementare */ }
   void stampa() { /*  da implementare */ }
} Tnodo;
```

[B - punti 1] *Modificare il file* ***main.cpp*** *includendo il file* ***dati.h*** *ed inserendo il seguente codice nel* ***main****:*

```
Tnodo* negozio[DIM];
for (int i=0; i<DIM; i++) { negozio[i] = NULL; }
Tacquisto o;
for (int i=0; i<5; i++) {newAcquisto(&o); addAcquisto(negozio, DIM, o); }
stampaNegozi(negozio, DIM);
cout << contaAcquisti(negozio, DIM, LIBRI);
```

[C - punti 4] Implementare una funzione **`newAcquisto`** che riceve in input l'indirizzo di una variabile di tipo **`Tacquisto`** **a** e la inizializza con:

- campo **`tipoProdotto`** valore casuale tra MUSICA, LIBRI, TECH;
- campo **`quantita`** valore letto da tastiera, tra 1 e 10, con controllo input.
- campo **`prezzo`** valore casuale tra 5.00 e 99.99;
- campo **`nomeProdotto`** letta da tastiera.

[D - punti 4] Implementare una funzione **`addAcquisto`** che riceve come input una variabile di tipo array **`Tnodo*`** **neg**, la dimensione dell'array **dim** e variabile di tipo **`Tacquisto`** **a** e la inserisce nella listaFIFO **neg[x]** (con **x** valore casuale tra 0 e 1) (**`insertFirst`**).

[E - punti 4] Implementare una funzione **`stampaNegozi`** che riceve in input una variabile di tipo array **`Tnodo`** *** neg**, la dimensione dell'array **dim** e stampa il contenuto delle liste FIFO utilizzando opportunamente il metodo **stampa di `Tnodo`**. Nella forma (per l'enum stampare etichetta in modo opportuno):

`nomeProdotto (tipoProdotto) QTA=quantita prezzo=prezzo`

[F - punti 5] Implementare una funzione **`contaAcquisti`** che riceve come input una variabile di tipo array **`Tnodo*`** **neg** e la dimensione dell'array **dim** e **`Tprodotto`** **tp** e conta tutti gli acquisti con tipoProdotto uguale a tp. La funzione, oltre a restituire il numero di questi acquisiti, aggiunge l'acquisito stesso ad un file denominato **`"selezioneAcquisti.txt"`** nella forma:

`nomeProdotto (tipoProdotto) QTA=quantita prezzo=prezzo`

È possibile creare strutture, funzioni, metodi aggiuntivi per realizzare gli esercizi proposti.
Il codice commentato NON verrà corretto!